

PROMPT-OPS

Pluggable Prompt Optimization SDK

Product Requirements Document

Version	1.0 — Draft
Author	Kaustubh
Date	May 2026
Status	In Review

1. Executive Summary

PROMPT-OPS is being evolved from a monolithic LLM observability system into a standalone, installable Python SDK that any developer can drop into their existing LLM application with a single decorator. The SDK owns the full optimization lifecycle — prompt versioning, A/B testing, temperature experiments, cost-aware model routing, automated quality evaluation, telemetry, and a self-contained dashboard — while the host application requires zero structural changes.

Core value proposition

Any developer adds two lines of code (@optimize decorator + pip install) and gets a closed-loop optimization engine that continuously improves their LLM application's quality and reduces API costs — without touching their existing code.

2. Problem Statement

2.1 The open-loop problem

The overwhelming majority of LLM applications are open-loop: a prompt goes in, a response comes out, and neither the developer nor the system has any structured feedback on whether that response was good. Developers are left guessing which prompt version works best, what temperature to use, and whether an expensive frontier model is even necessary for a given task.

2.2 Existing solutions fall short

Category	Tools	Gap
Observability only	LangSmith, Helicone, Arize	Track latency/cost, no quality loop

Category	Tools	Gap
Prompt management	PromptLayer, Agenta	Manual evaluation, no auto-optimization
Framework lock-in	LangChain callbacks, LlamaIndex	Requires full framework adoption
Research tools	DSPy, TextGrad	Compile-time, not runtime; complex setup

2.3 The integration tax

Current PROMPT-OPS itself has this problem: it is a monolithic application that a developer must adopt wholesale. There is no path to integrate just the optimizer into an app that already has its own LLM client, its own prompts, and its own infrastructure. The system needs to become a library, not an application.

3. Goals & Non-Goals

3.1 Goals

- Developer can install and integrate the optimizer with `pip install prompt-ops` and a single `@optimize` decorator — no other structural changes required.
- The SDK works with any LLM provider: OpenAI SDK, Anthropic SDK, custom HTTP clients, or LangChain chains.
- The optimizer owns its own storage (SQLite by default, PostgreSQL-ready), evaluation engine, and dashboard without depending on host app infrastructure.
- The closed feedback loop runs automatically: every decorated call is evaluated, and evaluation results feed back into future prompt and model selection decisions.
- First-run experience requires no configuration beyond an API key for the judge model; all defaults are sensible.
- The SDK is publishable to PyPI and installable in under 60 seconds.

3.2 Non-Goals

- PROMPT-OPS will not replace the host app's LLM client. It wraps it.
- PROMPT-OPS will not manage application-level auth, multi-tenancy, or user management.
- PROMPT-OPS will not perform compile-time prompt optimization (DSPy-style). Optimization is always runtime and incremental.
- PROMPT-OPS will not provide a hosted SaaS dashboard in this version. Dashboard runs locally alongside the application.
- PROMPT-OPS will not support streaming responses in v1. Streaming support is a v2 item.

4. Target Users

Persona	Context	Primary need
Solo developer	Side project or startup, one LLM feature	Zero-config integration, immediate value
Startup engineer	Production app, 1–3 LLM features, cost pressure	Cost routing + quality visibility
ML engineer	Multiple models, prompt experiments	A/B testing + temperature optimization
Final year student	Academic project, needs to demo results	Dashboard + metrics for presentation

5. Architecture Overview

5.1 Package boundary

The core design principle is a clean separation between the host application and the optimizer package. The host app calls its LLM function exactly as before. The decorator intercepts the call, runs the optimization pipeline, and returns the response transparently.

Host app owns	prompt-ops package owns
LLM client / SDK	Orchestrator & decorator
Application logic	Prompt versioning & A/B selector
UI / API layer	LLM-as-Judge evaluator
Business data	Temperature optimizer
Deployment infra	Cost router (tier cascade)
	Telemetry database (SQLite/Postgres)
	Streamlit dashboard

5.2 Integration interface

The public API surface is intentionally minimal:

Interface	Description
@optimize(prompt_id)	Decorator — wraps any function that calls an LLM
PromptOps.init(config)	One-time setup: DB path, judge model, API key
prompt_ops.create_version()	Register a new prompt version for A/B testing
prompt_ops.dashboard()	Launch the Streamlit dashboard

Interface	Description
OptimizeResult	Return type — wraps original response + metadata

5.3 Provider adapter pattern

The decorator does not make LLM calls itself. It calls the host's function and observes the result. This means it works with any provider without requiring an adapter for each one. The host function is the adapter.

6. Feature Requirements

6.1 Decorator & wrapper layer [P0]

- `@optimize(prompt_id, ab_testing=True, enable_cost_routing=False)` decorator that wraps any callable.
- Decorator must preserve the original function's signature, docstring, and return type annotation.
- Decorated function returns an `OptimizeResult` that exposes `.content` (original response), `.quality_score`, `.prompt_version`, `.model_used`, and `.cost_saved_usd`.
- If the optimizer fails for any reason, the decorator must fall back silently and return the original function's result — no exceptions should propagate to the host app.

6.2 Prompt versioning & A/B testing [P0]

- Developers register prompt versions with `create_version(prompt_id, template, name, traffic_weight)`.
- On each decorated call, the selector picks a version based on traffic weights using weighted random selection.
- Version metrics (request count, average quality, p95 latency) are updated after each call.
- The optimizer automatically promotes the highest-quality version to 100% traffic when it exceeds `AUTO_PROMOTE_THRESHOLD` (default 0.85) with statistical confidence.

6.3 LLM-as-Judge evaluation [P0]

- Every response is evaluated on five dimensions: relevance, accuracy, completeness, format compliance, and safety. Each scored 0.0–1.0.
- Evaluation uses a configurable judge model (default: `google/gemini-2.0-flash`) via OpenRouter.
- Evaluation is async by default and does not block the response returned to the host app.
- Evaluation sample rate is configurable (default 1.0; can be set to 0.1 for high-volume production use).

6.4 Temperature optimization [P0]

- `run_temperature_experiment(prompt_id, prompt_text, temp_min, temp_max, temp_step, trials_per_step)` runs a controlled sweep.
- Each temperature is scored on quality × consistency composite. Consistency = $1 - \text{std}(\text{quality_scores})$.
- The optimal temperature is stored and used automatically for all subsequent calls with that `prompt_id`.

6.5 Cost-aware model routing [P1]

- When `enable_cost_routing=True`, the router tries progressively stronger model tiers until quality threshold is met.
- Default tiers: free → cheap → mid → premium. All tiers are configurable via `MODEL_TIERS` in config.
- Routing decisions are logged with actual quality achieved, model used, and estimated cost saved.
- The router learns over time: if a prompt consistently passes quality checks on the free tier, it is pinned there until quality degrades.

6.6 Telemetry & monitoring [P0]

- Every decorated call is logged: timestamp, latency ms, input/output tokens, cost USD, model, prompt version, quality score.
- Monitor aggregates stats hourly and fires alerts when latency > 2000ms avg, error rate > 5%, or cost > \$10/day.
- Anomaly detection uses z-score (threshold: 2.0) on a rolling window of latency and cost.

6.7 Self-contained dashboard [P1]

- Dashboard is launched via `prompt_ops dashboard` or `PromptOps.dashboard()` — no separate process management required.
- Dashboard reads from the same SQLite database as the optimizer; no additional connection config needed.
- Dashboard pages: overview KPIs, model monitoring, prompt A/B results, quality evaluations, temperature experiments, cost routing, alerts, and settings.

6.8 Zero-config initialization [P0]

- On first import, the SDK checks for `PROMPT_OPS_API_KEY` (or `OPENROUTER_API_KEY`) in the environment and auto-initializes with defaults.

- SQLite database is created at `~/.prompt-ops/data.db` by default. Location is configurable.
- No `init()` call is required; decoration alone is sufficient for basic usage.

7. Non-Functional Requirements

Requirement	Target	Notes
Decorator overhead	< 2ms sync path	Async eval must not block return
Fallback reliability	100% — never raises	Catch-all on decorator outer layer
Install size	< 15 MB	No heavy ML dependencies
Python support	3.9, 3.10, 3.11, 3.12	Test matrix on all four
DB migration	Auto on version bump	Alembic or equivalent
Thread safety	Required	SQLAlchemy session per thread
Async support	v1: sync decorator only	Async decorator in v2

8. Technical Implementation Plan

8.1 Repository restructure

The current monorepo (`PROMPT-OPS/project/`) is split into two:

- `prompt-ops/` — the installable SDK package. Contains `src/`, `config/`, `dashboard/`, and `pyproject.toml`.
- `prompt-ops-demo/` — a thin demo application that depends on `prompt-ops` just like any other host app would. Contains `examples/` and `run.py`.

8.2 Package structure

Path	Purpose
<code>prompt_ops/__init__.py</code>	Public API: <code>optimize</code> , <code>PromptOps</code> , <code>OptimizeResult</code>
<code>prompt_ops/decorator.py</code>	Core <code>@optimize</code> implementation
<code>prompt_ops/orchestrator.py</code>	Pipeline: <code>select</code> → <code>call</code> → <code>evaluate</code> → <code>log</code>
<code>prompt_ops/evaluator.py</code>	LLM-as-Judge, 5 dimensions
<code>prompt_ops/optimizer.py</code>	A/B testing, version management, auto-promote
<code>prompt_ops/temperature.py</code>	Temperature sweep experiments
<code>prompt_ops/router.py</code>	Cost-aware tier cascade
<code>prompt_ops/telemetry.py</code>	Request logging, monitor, alerts
<code>prompt_ops/database/</code>	SQLAlchemy models + migrations

Path	Purpose
prompt_ops/dashboard/	Streamlit app (8 pages)
prompt_ops/config.py	Pydantic settings, env var loading

8.3 Decorator implementation strategy

The decorator uses a `functools.wraps` approach with a synchronous outer wrapper. The inner call flow is:

- Intercept arguments; extract the prompt string from the first positional arg or a named `prompt=` kwarg.
- Run prompt selector to pick the active A/B version and inject it if the function accepts a `prompt` parameter.
- Call the original host function with (potentially modified) arguments.
- Record telemetry start/end times and token counts from the response if available.
- Submit evaluation to a background thread pool — do not await.
- Return `OptimizeResult` wrapping the original response.

8.4 Dependencies

Core (required): `httpx`, `tenacity`, `sqlalchemy`, `pydantic-settings`, `loguru`.

Dashboard (optional, installed with `pip install prompt-ops[dashboard]`): `streamlit`, `plotly`.

Dev: `pytest`, `pytest-asyncio`, `black`, `ruff`, `mypy`.

9. Milestones & Delivery Plan

Milestone	Deliverable	Success criteria	Target
M1	Package scaffold + decorator	@optimize wraps any function, fallback tested	Week 1
M2	Evaluation + telemetry	Every call auto-scored and logged to DB	Week 2
M3	A/B testing + temp optimizer	Two prompt versions traffic-split, experiment runs	Week 3
M4	Cost router + monitoring	Tier cascade fires, alerts generated correctly	Week 4
M5	Dashboard + PyPI publish	pip install prompt-ops works, dashboard loads	Week 5
M6	Integration tests + docs	README quickstart works end-to-end in < 5 min	Week 6

10. Success Metrics

10.1 Integration experience

- Time from pip install to first decorated call producing a quality score: under 5 minutes.
- Lines of code the developer must change in their existing app: 2 (import + decorator).
- Decorator overhead on the synchronous call path: under 2ms measured in benchmarks.

10.2 Optimization effectiveness

- Over 50 calls, the auto-promoted prompt version achieves a higher average quality score than the initial default.
- Temperature optimizer selects a temperature that outperforms the default (0.7) in quality × consistency composite on at least 70% of experiment runs.
- Cost router reduces simulated API spend by 40% or more without dropping average quality below threshold.

10.3 Reliability

- Zero unhandled exceptions propagated to host app across 1000 decorated calls including injected LLM failures.
- Database recovers automatically after process restart with no data loss.

11. Risks & Mitigations

Risk	Impact	Mitigation
Judge model API key required	Blocks zero-config promise	Default to no evaluation if key absent; log a warning
Prompt extraction heuristic fails	Decorator cannot inject A/B version	Require explicit prompt= kwarg; document clearly
Evaluation latency blocks caller	Increases perceived latency	Evaluation is always async in background thread
SQLite write contention at scale	Data loss or corruption	WAL mode on; document Postgres path for > 50 RPS
LLM-as-Judge scores are biased	Bad optimization decisions	Expose raw dimension scores; let devs audit
Dashboard port conflicts	Dashboard fails to start	Auto-increment port; expose port= config option

12. Out of Scope (v2 Roadmap)

- Async decorator support for apps using asyncio or FastAPI.

- Streaming response evaluation — requires chunked output assembly before scoring.
- Bayesian optimization to replace grid search for temperature experiments.
- Hosted dashboard SaaS with multi-user support and role-based access.
- Webhook alerts (Slack, Discord, PagerDuty).
- Semantic similarity scoring using embeddings as a complement to LLM-as-Judge.
- Multi-turn conversation tracking across sessions.
- OpenTelemetry export for teams already using Grafana or Datadog.